

CSC_52089_EP Exam:

Final projects + Presentations

For your exam, we ask you to work on one of several suggested projects. Most of them build on the assignments of the class, extend them, and also require you to apply the implementations to data.

The key idea of these projects is to extend methods that you learned about in the class towards more realistic settings and/or work on topics that directly build on methods from the class.

Generally, we present you some core ideas for the projects along with ideas for extensions. We expect you to go beyond the *essential core* of the project, either following our suggestions or in other interesting ways.

You should work **in teams of two** (strongly recommended) on one of the suggested projects. Please talk to us, if you prefer to form teams of one or three. Each group should choose one of the projects from the list below and/or talk to us about specific variations.

The grading will be based on a presentation, your results, and your handling of questions after the presentation.

More concretely:

- We ask you to present the results in the **week of 16.03.2026**. (We plan to do the presentation online and will still coordinate the exact day and time with you.)
- The presentations should take: 20 min + 10/15 min (Q&A) per team of two (respectively, 30 min + 15/20 min (Q&A) for groups of three, or (15 min + 10 min (Q&A) for single students)
- Moreover, hand in a ‘report’ on your project (at least one day before the presentations). This ‘report’ must comprise:
 - your presentation slides
 - your code and brief documentation.
 - explanations and results (of your experiments in particular)

A Jupyter Notebook with careful explanation of experiments, plots, interpretations and empirical results can do the job. Alternatively, you could submit e.g. code + README + pdf with explanations, experiments and results.

The code should be reasonably commented, i.e. document classes, functions/methods, and their arguments. Moreover, provide sufficient instructions and examples such that we could run it. We also expect that in combination with the slides, your documents provide a brief overview and summary of what was done and some interpretation of your empirical results.

Important Notes

- *Experiments of your codes to get performances/empirical results* (and not just “toy examples”) are a mandatory part of the job and considered in the grading.
- The length of our project descriptions does not correlate with work load, the difficulty, or our expectations. Especially, you should always think about possible extensions yourselves, even if we don’t suggest specific extensions. We strongly advise you to go with the subjects that sound the more interesting to you/the one you feel the more in touch with!
- Beyond the project core, you are expected and as well free to expand each project the way you want. That means you should not feel limited to the suggested ways to go beyond the project core. On the contrary, we encourage you to come up with any original ideas and to implement and present them to us.

Please also feel free to discuss any details on the project by mail or in person.

Burrows-Wheeler index structures with partial tables

Subject

Implement a tool that implements the construction of a Burrows-Wheeler Transform (BWT) index and efficient search of words / reads with the help of the index. Here, we want you to implement a variant that can find occurrences in $O(k)$ time, where n is the text length and k is the word length. At the same time, your implementation should have a low memory footprint.

Core requirements

Recall that from the BWT alone one cannot search efficiently in linear or even logarithmic time (w.r.t. text length). Another problem is that without additional tables, one cannot efficiently report the positions where the pattern occurs. As we discussed in the class, these issues can be overcome by additional tables.

- Your implementation is supposed to make use of additional tables to speed up the search, such that you can search in $O(k)$ time (linear in the pattern length). As core requirement, it suffices to report the number of occurrences as result of the search.
- Improve the memory footprint by storing the required tables only partially. Measure and/or calculate the space improvements.
- Test on medium-sized genomes, e.g. RNA viruses (HIV, SARS-Cov2, ...) and map simulated reads without errors.
- Collect and present statistics that demonstrate the asymptotic time and space behavior of your implementation.

To go beyond

- Study the space-vs-time trade off due to storing only every K -th row of the tables.
- Extend your method by an partially stored suffix array, such that also the positions of occurrences can be reported. Note that when storing only every K -th entry of the SA, the missing entries can be must be recomputed. The can be recalculated very efficiently by using the other tables of the BWT. By going backward until a stored SA entry is discovered (using the last-first-mapping, which can be computed with the help of count tables).
Study the performance impact of adding the partial SA and reporting occurrence positions.
- Extend your search algorithm to allow Δ many mismatches. Test your algorithm, provide example results and performance measurements.

Hints

- Recall from the lecture and the index structure assignment that you already implemented a time-inefficient variant of mapping with the BWT index.
- Note that we don't require you to implement more efficient methods for the *construction* of the index.

Exact multiple sequence alignment with bounding

Subject

Multiple sequence alignment asks for an optimal alignment of K sequences. Here, we ask you to implement exact algorithms to solve the problem. Again, You have seen elementary definitions and an instance of an exact dynamic programming algorithm by DP in Lecture 04.

We ask you to maximize similarity, where we can define the similarity of an alignment column as sum-of-pair based on a pairwise similarity table $s(x,y)$ for pairs of all alphabet characters (e.g. nucleotides or amino acids) and the gap symbol. Then, the sum-of-pairs similarity of a column $x_1 \dots x_K$ is $s(x_1, \dots x_K) = \sum_{1 \leq k < \ell \leq K} s(x_k, x_\ell)$.

Core requirements

- Implement a dynamic programming algorithm that find the exact maximum similarity of any MSA for K input sequences (arbitrary $K \geq 2$). This algorithm should be a direct extension of the pairwise dynamic programming algorithm as shown in Lecture 4 for $K = 3$. (While a generic implementation for arbitrary K is strongly preferred, you may want to start with implementations for $K = 3$ and $K = 4$.)

Test this only for small instances. We do not expect this to scale very well!

- Develop an improved algorithm using Carillo-Lipman bounds.

Carrillo-Lipman bounds are a heuristic used in multiple sequence alignment (MSA) to reduce the search space and improve efficiency. The key idea is to leverage pairwise alignment scores to compute only a fraction of the matrix entries.

We provide the essential details below. You might be also interested in the original paper <https://epubs.siam.org/doi/10.1137/0148063> or find details in Dan Gusfield's bioinformatics text book "Algorithms on Strings, Trees and Sequences".

- Compare the two algorithms on examples. They should compute the exactly same optimal alignment scores. Run the algorithms on examples that convincingly test the correctness of your implementations. Think about covering critical special cases etc. You should observe perfect agreement, but if you cannot see this, track down the issues as much as possible.
- Obtain optimal MSAs by backtracking.

To go beyond

- Study and compare the performance of the algorithms. Estimate the performance of the non-bounded algorithm. Study the effect of different bounds.
- How do exact alignments relate to heuristic alignments. Compare to available tools or your own implementation. Quantify and discuss differences.
- Use reasonable similarity matrices like BLOSUM62 and find interesting instances.
- Can you overcome any limitations of the bounding approach?

Carillo-Lipman bounds

1. **Pairwise Alignment Scores** As precomputation, calculate pairwise alignments between all sequences. Specifically, we need similarities between all **suffix** sequences. (One could rewrite Needleman-Wunsch or reverse the sequences.) Define $S_{k,\ell}(i,j)$ as the similarity between the suffixes of sequence k and ℓ respectively starting with i and j .
2. **Bounding** The sum of all pairwise alignment similarities yields an upper bound on the total MSA similarity. Similarities of MSAs of suffixes, which complete MSAs of prefixes to total MSAs, can be bounded following the same logic.
3. **Pruning in the computation of MSA similarity** We run our MSA algorithm for a specific bound z . Based on this bound, we modify the

exact algorithm, such that we ask for the exact optimal similarity **if** it is at least z . Otherwise, we learn that the similarity is smaller. The higher we choose z , the faster we can compute this.

Our MSA algorithm calculates prefix MSA similarities $S(i_1, \dots, i_K)$ for the prefixes of the sequences up to respective $i_1, \dots, i_K$. The pairwise suffix scores let us calculate an upper bound on the best score obtainable from entry $(i_1, \dots, i_K)$. If this bound is lower than z , then the entry is not required to find the optimal similarity.

The key to improved performance is now to exploit this information, such that the algorithm can rule out the computation of as many matrix entries as possible. For this purpose, the algorithm can maintain a queue of entries that must be calculated (since they can lead to the optimal total solution).

Large parsimony with Quartet Puzzling

Subject

In the class, you developed an algorithm to solve the small parsimony problem given a set of sequences and addressed large parsimony based on nearest neighbor exchange.

In this project, we suggest an alternative idea to solve the large parsimony problem based on combining solutions for quartets.

Such a strategy has been suggested by Strimmer and Haeseler as Quartet Puzzling Algorithm: <https://academic.oup.com/mbe/article/13/7/964/1047687>. While this strategy was originally proposed to combine maximum likelihood trees of sequence quartets, one can equally well use it to combine maximally parsimonious quartet trees.

Core requirements

- Given a set of K sequences, determine the large parsimony solutions for all 4-tuples of sequences (yielding the quartet trees). Combine the quartet trees into a single good tree for the K sequences following ideas from the paper of Strimmer and Haeseler (and/or your own ideas). Note that Strimmer and Haeseler suggest a puzzling **and** a consensus step (Margush&Morris, 1981, Consensus n-trees, <https://link.springer.com/article/10.1007/BF02459446>).
- Find reasonable ways to generate input instances and/or find real data.
- Measure the performance of the optimization strategy, compare the parsimony scores of your solutions to other strategies. For example,
 - 1) determine the scores on UPGMA trees or neighbor joining trees or
 - 2) find solutions by nearest neighbor interchange as in Assignment 05.

When comparing non-deterministic strategies, remember to rerun them several times to ensure reasonable robustness of their results.

To go beyond

- Test different variations and/or parametrizations of your quartet puzzling strategy.
- Perform more tests to obtain conclusive results.
- Demonstrate and visualize your results.

Suboptimal expected accuracy structures of RNAs

Subject

In addition to finding the best RNA structures, it is oftentimes informative to predict the close to optimal structures. Here, we ask you to enumerate RNA structures with suboptimal expected accuracies (within a certain range of the optimal solution).

This project requires you to cleanly implement suboptimal traceback as discussed in the assignments of class RNA-I.

We compute energies in a model that is typically used to compute maximum expected accuracy (MEA) structures; therefore, call them EA suboptimals. This is more realistic than the base pair maximization model, since it evaluates structures based on base pair probabilities computed by the McCaskill algorithm from the Vienna RNA package. Details are described below.

Core requirements

- Implement the computation of structures with sub-optimal expected accuracy.
 - Construct a pseudo-energy function from the base pair probabilities p_{ij} , as calculated from this sequence by the Vienna RNA package (`RNAfold -p`; respectively Python functions `RNA.pf()` and `RNA.bpp()`).
 - Then, assign the pseudo energy $E_{ij} = -2 * \gamma * p_{ij}$ to each base pair (i, j) and $E_i^u = -(1 - \sum_j p_{ij})$, i.e. the negated probability that i is not paired, to each unpaired base i . (γ is a weight, e.g. $\gamma = 1$; explain and demonstrate its effect).
- Study properties of the EA suboptimals, e.g. the relation between their EA and their base pair counts, Turner energies (as computed by the RNA package, `RNA.fold_compound.eval_structure`), ...
- Demonstrate some differences between EA suboptimals and Turner energy suboptimals (Vienna RNA package)

To go beyond

- Elaborate on your study showing interesting differences between the EA suboptimal structures and the ones based on the Turner energy model (as computed by RNAsubopt) for a set of real RNAs from different Rfam families or the RNAstrand database. For example, distributions of energies over suboptimals in both energy models; correlations; sum of Boltzmann weights (fraction of the total partition functions), ...

Sampling and base pair probabilities of RNA structures

Subject

Similar to enumerating suboptimal structures, sampling of good structures can provide useful insights into the structure potential of an RNA. Here, we want to sample RNA structures from Boltzmann distributions with regard to expected accuracy.

Core requirements

- Come up with an algorithm that samples structures from their Boltzmann distribution. That is, it generates structures with probability proportional to their Boltzmann weight. This should be based on the partition function algorithm for the input sequence S and a specific base pair energy.

Your algorithm should define Boltzmann weights based on expected accuracies, compare use of expected accuracy in other projects and see details below.

- Implement and apply the algorithm to different RNA sequences (include sequences of lengths at least greater than 50).
- Plot histograms of the energies of the sampled structures for different β . Find interesting values of β . Which effects of β do you observe...
- Estimate the probabilities of base pairs and visualize them in the form of a dot plot. How do these probabilities compare to the probabilities predicted by the Vienna RNA package. Optionally, do the same for stacks that is pairs of base pairs (i,j) and $(i+1,j-1)$.

To go beyond

- Demonstrate and discuss applications of such a sampling algorithm. For example: find other interesting quantities that can be estimated from the sampling (e.g. average number of base pairs...); estimate them.

Details / Hints

Start from an algorithm for computing partition functions for secondary structure.

Use a base pair-based energy model that evaluates structures R by expected accuracy, i.e. we define the energy $E(R) = -(\sum_{(i,j) \in R} 2^{\gamma} p_{ij} + \sum_{i \text{ not paired in } R} p_i^u)$.

Your partition functions should sum over Boltzmann weights $\exp(-\beta E(R))$, where β is a real-valued parameter, which controls the corresponding Boltzmann distribution.

A good start into the project could be to first consider only uniform sampling based on a counting algorithm of RNA structures. The Boltzmann sampling can be seen as a strict generalization of uniform sampling.

The uniform algorithm has the following idea, where the Boltzmann sampling works analogously using partition functions in place of counts: a sample is generated by a traceback algorithm, often called *stochastic traceback*. When tracing back in the matrix from (i, j) , we randomly choose one of the cases with a probability proportional to the number of structures (the partition function) corresponding to this case.

Clustering of sampled RNA structures

Subject

The Vienna RNA package provides the capability to generate/sample RNA structures with their probabilities in the thermodynamic equilibrium (i.e. Boltzmann probabilities based on the free energies). The package does this by stochastic traceback after computing the partition function (McCaskill algorithm). Sampling of structures allows us to study clusters in the structure space.

Core requirements

- Use the Vienna package to generate Boltzmann-distributed random structures. For a set of real RNAs and possibly a few self-designed artificial ones, determine base pair probabilities from the sampled RNAs and visualize them as dot plots. Match your result with the base pair probabilities that are calculated by the Vienna RNA package. Produce at least 1000 samples per RNA (can you say anything about convergence?).
- Come up with ways to cluster the sampled structures. The clustering should be based on a structural distance, e.g. their base pair distance or hamming distance of their dot bracket strings. One could apply dimensionality reduction techniques and plot the result. Apply k-means clustering or other sufficiently fast methods of your choice to determine clusters. Again cluster at least several hundreds of samples per RNA.
- Estimate cluster probabilities. Then, calculate base pair probabilities in the single clusters and visualize them for the most prominent clusters. Can you find good examples (natural and possibly artificial RNAs) that show several prominent clusters. Can the total dot plot be explained in terms of the cluster dot plots? Compare the paper on S-fold:

<https://pmc.ncbi.nlm.nih.gov/articles/PMC1370799/>. Can you obtain a visualization of clusters comparable to Figure 1?

- Compute representative structures of each cluster. In the simplest case, determine cluster centroid structures by selecting all base pairs with more than 50% conditional probability under the condition that the structure is in this cluster. This will always produce a valid structure, why?

To go beyond

- Beyond base pair probabilities, what else could be learned from the clusters? (diversity of cluster ensembles, ...). Determine some measures. Compare different RNAs. Think about visualizations.
- Refine the idea of cluster representatives. Instead of determining a centroid structure, compute MEA structures as cluster representatives. Compare results of different ideas / parameters. MEA structures can be calculated using a Nussinov type of folding algorithm (see the project on suboptimal expected accuracy structures.)
- Study convergence behavior. Does the number of samples for convergence depend on sequence length?

Hints

- This project should use the sampling functionality of the Vienna package, which is available as method `fc.pbacktrack()` of the fold compound `fc` after calling `fc.pf()`. (This is different to the project on sampling of RNA structures, where the idea is to implement a sampling algorithm.)
- Computing MEA (maximum expected accuracy) structures: a MEA structure R has maximal expected accuracy $EA(R) = \sum_{(i,j) \in R} 2 * \gamma * p_{ij} + \sum_{i \text{ not paired in } R} p_i^u$. We can maximize the EA, and thereby finally compute an MEA structure, by a Nussinov like algorithm that adds the unpaired probability p_i^u in the case, where the base i is unpaired and $2 * \gamma * p_{ij}$, in the cases where (i, j) is paired. Here, $p_i^u = 1 - \sum_j p_{ij}$.

Determine energy barriers in RNA folding landscapes

Subject

RNA structure prediction, as we discussed it in class, asks for global optima in the space of possible structures. We have also seen the computation of structures close to these optima. These algorithms ignore that the folding of RNAs takes a certain time; instead, they just provide us with a static picture of the dynamic folding process. Note that even McCaskill's algorithm, lets us compute probabilities only at thermodynamic equilibrium, i.e. after infinite time.

The time-dependent dynamic behavior of RNAs can be modeled based on elementary changes of RNA secondary structures, e.g. base pair moves, which close or open one base pair at a time. We can then ask whether two RNA structures can be connected by energetically favorable paths.

Given an RNA sequence s , two valid secondary structures L and R of s and a function E computing the energy of a sequence over a structure, we define a reconfiguration path between L and R of barrier k as a sequence of l non-crossing secondary structures $S_0 = L, \dots, S_l = R$ such that

- $S_0 = L$ and $S_l = R$;
- $E(s, S_i) - E(s, L) \leq k, \forall i \in [0, l]$;
- $|S_i \Delta S_{i+1}| = 1, \forall i \in [0, l - 1]$.

Informally, S_i and S_{i+1} differ by exactly one base pair (i.e. they are connected by an elementary base pair move). The highest energy on an optimal pathway of reconfiguration is called *barrier* and corresponds to (the smallest) k in the above problem statement.

The task is to find a path with a minimum barrier between two structures. As the problem is known to be NP-complete even in a base pair energy model, it makes sense to rely on heuristics to find small barriers.

Core requirements

- Implement direct path heuristics to find a path between two given structures with small barrier. First, implement this for a base pair energy model, where every base pair has energy -1.
- Then, extend this beyond base pair energy by evaluating the energy of a structure as Turner energy using the Vienna RNA package (`fold_compound.eval_structure`). In the following use this more realistic energy function.

Such heuristics consider only direct paths between the structures L and R , i.e. on the pathway the distance to L is strictly increasing, while the distance to R is decreasing. Typically, a direct path is only made of removed base pairs from L and added base pairs from R .

- Propose a heuristic to compute paths in a version of the problem that includes non-direct paths. For instance, one can greedily build the path from energetically best continuations.
- Propose bounds of complexities (even if exponential).
- Study the results from your heuristics on examples.

To go beyond

- Heuristic smarter variants: explore all possibilities or keep a front of k-best structures in their exploration. . .
- Find examples, where the direct path or other strategies fail.
- Similarly, one can observe some “favorable” cases in terms of time of computations. In that case depicts what is special with these cases.

Hints

- If you need more ideas or a starting idea, one could look up Morgan-Higgs heuristic for the barrier computation.